

# OpenShift Cluster Observability

Building Custom Perspectives Dashboards for VMs and Microservices

*Internal runbook — Platform Engineering*

# 1. Overview

This guide walks through configuring the OpenShift Cluster Observability Operator (COO) to build custom Perses dashboards backed by a global Thanos Querier datasource. It covers two example dashboards: one for monitoring OpenShift Virtualization (KubeVirt) VMs, and one for monitoring a sample microservice application.

**Scope:** This document focuses on custom dashboards via Perses and a shared Thanos datasource. It does not cover distributed tracing or automated incident detection — if those are needed, they should be documented separately.

## 2. Prerequisites

- The OpenShift Cluster Observability Operator is deployed on the cluster.
- A user account with cluster administrator privileges.
- OpenShift Virtualization is deployed, with one or more VMs running (required only for the VM dashboard in Part 2).
- One or more microservices are running (required only for the application dashboard in Part 3).

## 3. Part 1 — Enable Perses and Configure the Global Datasource

These steps enable the Perses web UI shipped with the Cluster Observability Operator and connect it to Thanos Querier as a shared, cluster-wide datasource that any dashboard can reuse.

## 3.1 Enable the Perses web UI

Enable the Perses UI plugin in the openshift-cluster-observability-operator namespace by applying:

```
apiVersion: observability.openshift.io/v1alpha1
kind: UIPlugin
metadata:
  name: monitoring
spec:
  type: Monitoring
  monitoring:
    perses:
      enabled: true
```

Once applied, confirm the perses and perses-operator pods are created and running:

```
[lab-user@bastion ~]$ oc get pods -n openshift-cluster-observability-operator
NAME                                                    READY   STATUS    RESTARTS   AGE
monitoring-79bc79c564-wj5dr                            1/1     Running   0           29m
obo-prometheus-operator-55559d8fc9-vfxwc               1/1     Running   0           31m
obo-prometheus-operator-admission-webhook-f796d478-9ps8v 1/1     Running   0           31m
obo-prometheus-operator-admission-webhook-f796d478-jrf8s 1/1     Running   0           31m
observability-operator-6bb886b9-mrwnt                   1/1     Running   0           31m
perses-0                                                 1/1     Running   0           29m
perses-operator-55bb94b7c9-8mtm2                       1/1     Running   0           31m
[lab-user@bastion ~]$
```

Figure 1 — perses and perses-operator pods running

## 3.2 Create a service account for Thanos queries

Create a dedicated service account that Perses will use to query Thanos:

```
oc create serviceaccount perses-thanos-reader \
-n openshift-cluster-observability-operator
```

## 3.3 Grant monitoring permissions

Bind the cluster-monitoring-view role to the service account so it can read metrics:

```
oc adm policy add-cluster-role-to-user cluster-monitoring-view \ -z perses-thanos-reader \
-n openshift-cluster-observability-operator
```

## 3.4 Deploy the global datasource

Create a PersesGlobalDatasource pointing at Thanos Querier. Because it's global, any dashboard in the cluster can reference it by name:

```

apiVersion: perses.dev/v1alpha2
kind: PersesGlobalDatasource
metadata:
  name: thanos-querier-datasource
spec:
  config:
    display:
      name: "Thanos Querier Datasource"
      default: true
    plugin:
      kind: "PrometheusDatasource"
    spec:
      proxy:
        kind: HTTPProxy
        spec:
          url: https://thanos-querier.openshift-monitoring.svc.cluster.local:9091
          secret: thanos-querier-datasource-secret
  client:
    tls:
      enable: true
      caCert:
        type: file
        certPath: /ca/service-ca.crt

```

After applying the YAML, verify the datasource resource was created:

```

[lab-user@bastion ~]$ oc get persesglobaldatasources
NAME                                AGE
thanos-querier-datasource          25m
[lab-user@bastion ~]$

```

Figure 2 — *thanos-querier-datasource* global datasource created

## 4. Part 2 — VM Monitoring Dashboard

With the global datasource in place, create a dashboard that monitors OpenShift Virtualization VMs. This can be done either from the console or by applying YAML directly.

### 4.1 Create the dashboard

- From the console: navigate to Observe → Dashboards (Perses), click Create in the top right, and give the dashboard a name and namespace.
- Or apply the following PersesDashboard YAML directly:

```

apiVersion: perses.dev/v1alpha2
kind: PersesDashboard
metadata:
  name: vmdb2
  namespace: openshift-cluster-observability-operator # <-- replace with your Perses
instance's actual namespace
spec:
  config:
    display:
      name: VM Dashboard
      duration: 5m
      refreshInterval: 5s
      variables: []
      datasources: {}
      panels:
        4863e7a7c6a34338a6131ba451ac529a:
          kind: Panel
          spec:
            display:
              name: Number of VM running (in %)
            plugin:
              kind: StatChart
              spec:
                calculation: last-number
                format:
                  unit: decimal
                legendMode: auto
                sparkline: {}
            queries:
              - kind: TimeSeriesQuery
                spec:
                  plugin:
                    kind: PrometheusTimeSeriesQuery
                    spec:
                      datasource:
                        kind: PrometheusDatasource
                        name: thanos-querier-datasource
                        minStep: 5s
                        query: |-
                          count(kubevirt_vm_info{status="running"})
                          /
                          count(kubevirt_vm_info) * 100
                      seriesNameFormat: Total VMs
        78a6fe01c2ed486d84f086e3142da861:
          kind: Panel
          spec:
            display:
              name: "kubevirt CDI CR status "
            plugin:
              kind: StatChart
              spec:
                calculation: last-number
                format:
                  unit: decimal

```

```

        legendMode: auto
        sparkline: {}
    queries:
        - kind: TimeSeriesQuery
          spec:
            plugin:
              kind: PrometheusTimeSeriesQuery
            spec:
              datasource:
                kind: PrometheusDatasource
                name: thanos-querier-datasource
                query: kubevirt_cdi_cr_ready
90a033f5a00841eead352a968944840a:
kind: Panel
spec:
display:
  name: "Free memory of all VMs "
plugin:
  kind: Table
  spec:
    cellSettings: []
    columnSettings:
      - hide: true
      name: timestamp
      - name: namespace
      - name: pod
      - format:
          unit: decimal
      name: value
      density: standard
      enableFiltering: true
    queries:
      - kind: TimeSeriesQuery
        spec:
          plugin:
            kind: PrometheusTimeSeriesQuery
          spec:
            datasource:
              kind: PrometheusDatasource
              name: thanos-querier-datasource
              query: sum by (namespace, pod)
                (kubevirt_vm_container_free_memory_bytes_based_on_rss) /
                1024 / 1024 / 1024
d981bf70668a4b09aad16204cd6a4f12:
kind: Panel
spec:
display:
  description: OS/Status/Namespcae
  name: "VMs List "
plugin:
  kind: Table
  spec:
    columnSettings:
      - hide: true

```

```
      name: timestamp
      - hide: true
      name: value
      density: standard
      enableFiltering: true
queries:
  - kind: TimeSeriesQuery
    spec:
    plugin:
    kind: PrometheusTimeSeriesQuery
    spec:
      datasource:
      kind: PrometheusDatasource
      name: thanos-querier-datasource
      query: count by (status, os, namespace) (kubevirt_vm_info)
layouts:
  - kind: Grid
spec:
display:
  title: Panel Group
  collapse:
  open: true
items:
  - x: 0
    "y": 0
    width: 10
    height: 7
    content:
    $ref: "#/spec/panels/4863e7a7c6a34338a6131ba451ac529a"
  - x: 0
    "y": 7
    width: 24
    height: 8
    content:
    $ref: "#/spec/panels/d981bf70668a4b09aad16204cd6a4f12"
  - x: 10
    "y": 0
    width: 14
    height: 7
    content:
    $ref: "#/spec/panels/78a6fe01c2ed486d84f086e3142da861"
  - x: 0
    "y": 15
    width: 24
    height: 8
    content:
    $ref: "#/spec/panels/90a033f5a00841eead352a968944840a"
links: []
```

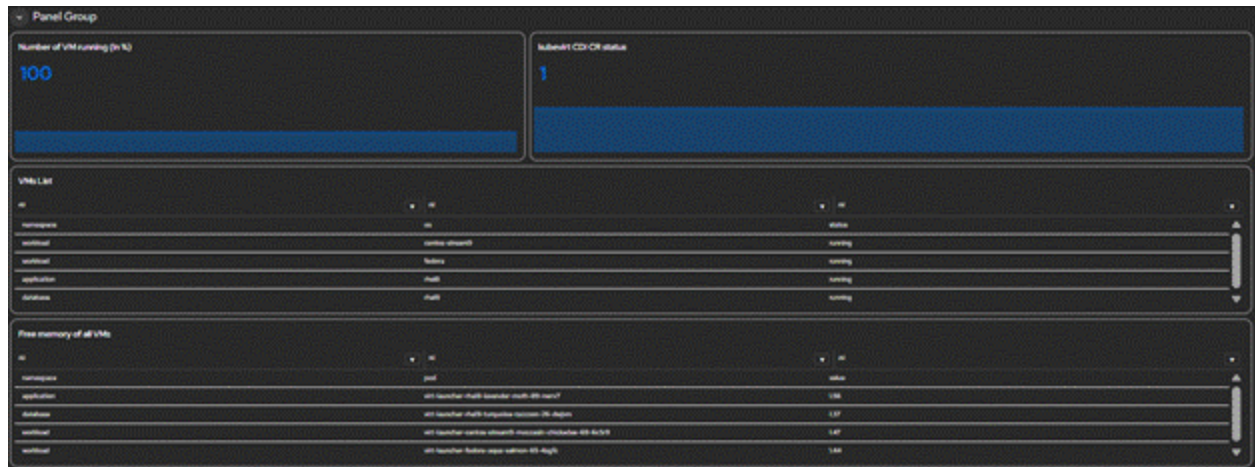


Figure 3 — Resulting VM dashboard in the Perses UI

## 5. Part 3 — Microservice Application Dashboard

This example dashboard monitors a sample online sock shop application composed of the following microservices:

- Carts
- Catalogue
- Catalogue-db
- Front-end
- Orders
- Orders-db
- Payment
- Queue-master
- Rabbitmq
- Shipping
- User
- User-db

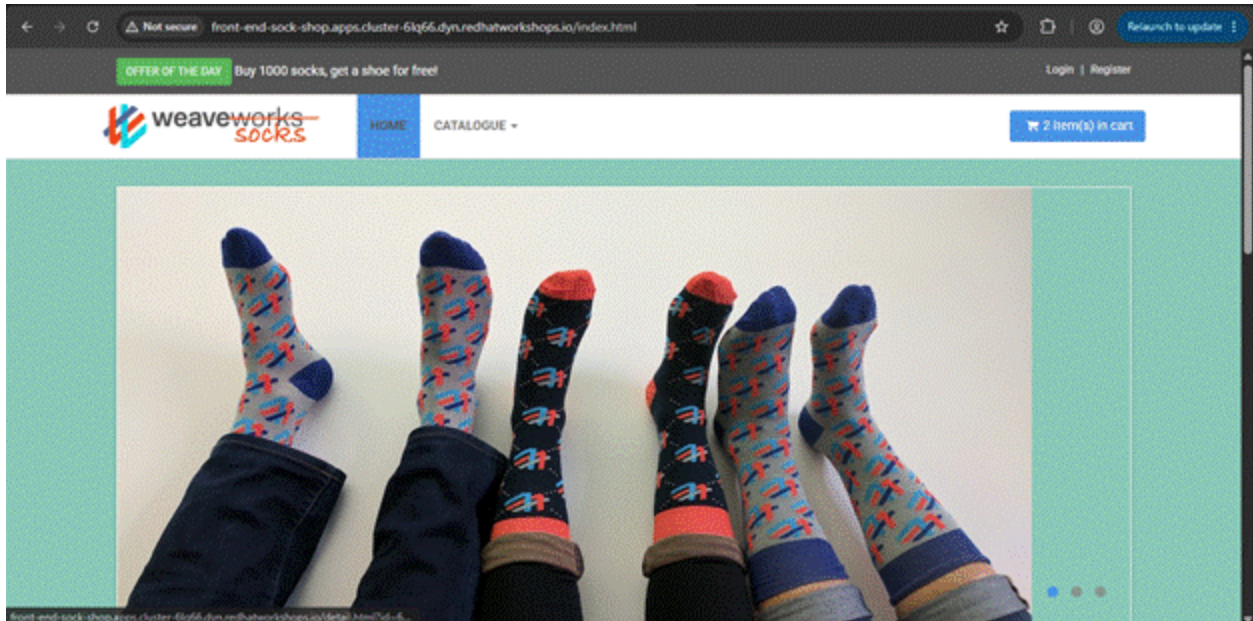


Figure 4 — Sample sock shop application UI

## 5.1 Enable User Workload Monitoring

User Workload Monitoring (UWM) must be enabled so Prometheus scrapes metrics from application namespaces:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

## 5.2 Create a ServiceMonitor

Add a ServiceMonitor for the front-end service:

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: front-end
  namespace: sock-shop
  labels:
    app: sock-shop
spec:
  selector:
    matchLabels:
      name: front-end
  endpoints:
    - port: web
      path: /metrics
      interval: 15s
```

## 5.3 Create a PodMonitor

Add a PodMonitor covering the remaining application pods:

```
apiVersion: monitoring.coreos.com/v1
kind: PodMonitor
metadata:
  name: sock-shop
  namespace: sock-shop
  labels:
    app: sock-shop
spec:
  selector:
    matchExpressions:
      - key: name
        operator: In
        values:
          - carts
          - catalogue
          - front-end
          - orders
          - payment
          - queue-master
          - shipping
          - user
  podMetricsEndpoints:
    - port: metrics
      path: /metrics
      interval: 15s
```

## 5.4 Create the application dashboard

**Important:** The YAML and panel layout below are tailored to this sample application's metric names. Adjust queries and labels to match the metrics actually exposed by your own microservices.

Apply the following PersesDashboard YAML, or use the metrics reference in section 5.5 to build equivalent panels from the console:

```

apiVersion: perses.dev/v1alpha2
kind: PersesDashboard
metadata:
  labels:
    app.kubernetes.io/name: perses-dashboard
    app.kubernetes.io/instance: application
    app.kubernetes.io/part-of: perses-operator
  name: test-app-dashboard
  namespace: sock-shop
spec:
  config:
    display:
      name: Application
    panels:
      2afe4ac11b0c40ffb09b09dd5b82328a:
        kind: Panel
        spec:
          display:
            name: Request rate per microservice
        plugin:
          kind: Table
          spec:
            density: standard
            enableFiltering: true
            columnSettings:
              - name: value
              hide: true
              - name: namespace
              - name: path
              - name: pod
              - name: status_code
              - name: timestamp
            format:
              unit: relative-time
        queries:
          - kind: TimeSeriesQuery
            spec:
              plugin:
                kind: PrometheusTimeSeriesQuery
              spec:
                query: >-
                sum by (namespace, path, pod, status_code) (
                  http_request_duration_seconds_count{namespace="sock-shop"}
                )
                datasource:
                  kind: PrometheusDatasource
                  name: thanos-querier-datasource
      424543212c9a47338d1377c4b34bfbac:
        kind: Panel
        spec:
          display:
            name: "Total requests "
            description: total cumulative number of HTTP requests recorded
        plugin:

```

```

        kind: StatChart
        spec:
          calculation: last-number
          format:
          unit: decimal
          sparkline: {}
          legendMode: auto
      queries:
        - kind: TimeSeriesQuery
          spec:
            plugin:
              kind: PrometheusTimeSeriesQuery
            spec:
              query: >-
                sum(
                  http_request_duration_seconds_count{namespace="sock-shop"}
                )
              datasource:
                kind: PrometheusDatasource
                name: thanos-querier-datasource
6d648fbd3d424e4f98bf2848f5e15f5b:
kind: Panel
spec:
display:
  name: request latency
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
      plugin:
        kind: PrometheusTimeSeriesQuery
      spec:
        query: >-
          1000 *

          (
            sum by (path) (

rate(http_request_duration_seconds_sum{namespace="sock-shop"}[5m])
            )
          /
            sum by (path) (

rate(http_request_duration_seconds_count{namespace="sock-shop"}[5m])
            )
          )
          and

            sum by (path) (

rate(http_request_duration_seconds_count{namespace="sock-shop"}[5m])

```

```

        ) > 0
        datasource:
        kind: PrometheusDatasource
        name: thanos-querier-datasource
57c197a19f4f4d1e81cd4af1631ee33c:
kind: Panel
spec:
display:
    name: P95 latency
plugin:
    kind: TimeSeriesChart
    spec: {}
queries:
    - kind: TimeSeriesQuery
      spec:
        plugin:
        kind: PrometheusTimeSeriesQuery
        spec:
            query: >-
            1000 * histogram_quantile(
            0.95,
            sum by (path, le) (
rate(http_request_duration_seconds_bucket{namespace="sock-shop"}[5m])
            )
            )
        datasource:
        kind: PrometheusDatasource
        name: thanos-querier-datasource
cf6bb5ba1541485db8ca61e255181258:
kind: Panel
spec:
display:
    name: P99 latency
plugin:
    kind: TimeSeriesChart
    spec: {}
queries:
    - kind: TimeSeriesQuery
      spec:
        plugin:
        kind: PrometheusTimeSeriesQuery
        spec:
            query: >
            1000 *

            histogram_quantile(
            0.99,
            sum by (path, le) (
rate(http_request_duration_seconds_bucket{namespace="sock-shop"}[5m])
            )
            )
        datasource:

```

```
      kind: PrometheusDatasource
      name: thanos-querier-datasource
8de86b66dc7641d8ac278acf175e3617:
kind: Panel
spec:
display:
  name: Pod UP/DOWN
plugin:
  kind: Table
  spec:
    density: standard
    enableFiltering: true
    columnSettings:
      - name: timestamp
    hide: true
queries:
  - kind: TimeSeriesQuery
    spec:
      plugin:
        kind: PrometheusTimeSeriesQuery
        spec:
          datasource:
            kind: PrometheusDatasource
            name: thanos-querier-datasource
          query: |-
            count by (pod) (
              up{namespace="sock-shop"}
            )
3198885c174d49c990e238f64d2ec2dc:
kind: Panel
spec:
display:
  name: "CPU used per service "
  description: This is CPU cores being consumed, approximately.
plugin:
  kind: TimeSeriesChart
  spec:
    querySettings: []
queries:
  - kind: TimeSeriesQuery
    spec:
      plugin:
        kind: PrometheusTimeSeriesQuery
        spec:
          query: >
            sum by (service) (
              rate(process_cpu_seconds_total{namespace="sock-shop"}[5m])
            )
          datasource:
            kind: PrometheusDatasource
            name: thanos-querier-datasource
links: []
d00a3bbc94644c11a7c442be5068ea66:
kind: Panel
```

```
spec:
display:
  name: Process memory per service
  description: Process memory per service in GB
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
    plugin:
    kind: PrometheusTimeSeriesQuery
    spec:
      query: |-
        sum by (service) (
          process_resident_memory_bytes{namespace="sock-shop"}
        ) / 1024^3
      datasource:
        kind: PrometheusDatasource
        name: thanos-querier-datasource
dc089e5df86140b09d0e9cda44ce17c8:
kind: Panel
spec:
display:
  name: JVM memory used
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
    plugin:
    kind: PrometheusTimeSeriesQuery
    spec:
      query: |-
        sum by (container) (
          jvm_memory_bytes_used{namespace="sock-shop"}
        )
      datasource:
        kind: PrometheusDatasource
        name: thanos-querier-datasource
fe9b2a5b8d1843e4af6851f017b8e133:
kind: Panel
spec:
display:
  name: JVM threads
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
    plugin:
    kind: PrometheusTimeSeriesQuery
```

```

      spec:
        query: |-
          sum by (container) (
            jvm_threads_current{namespace="sock-shop"}
          )
        datasource:
          kind: PrometheusDatasource
          name: thanos-querier-datasource
38724d4938f34688b3cfe8348f592798:
kind: Panel
spec:
display:
  name: JVM GC rate
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
      plugin:
        kind: PrometheusTimeSeriesQuery
        spec:
          query: >-
            sum by (container) (
              rate(jvm_gc_collection_seconds_count{namespace="sock-shop"}[5m])
            )
          datasource:
            kind: PrometheusDatasource
            name: thanos-querier-datasource
e8d03e21e9894b0e923485ac55b5c0ef:
kind: Panel
spec:
display:
  name: Node.js active requests
plugin:
  kind: StatChart
  spec:
    calculation: last-number
    format:
      unit: decimal
    sparkline: {}
    legendMode: auto
queries:
  - kind: TimeSeriesQuery
    spec:
      plugin:
        kind: PrometheusTimeSeriesQuery
        spec:
          query: |-
            sum by (container) (
              nodejs_active_requests_total{namespace="sock-shop"}
            )
          datasource:
            kind: PrometheusDatasource

```

```
      name: thanos-querier-datasource
57f70808f6fe488cb3082c81a508f583:
kind: Panel
spec:
display:
  name: Node.js event loop lag
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
    plugin:
    kind: PrometheusTimeSeriesQuery
    spec:
      query: |-
        max by (service) (
          nodejs_eventloop_lag_seconds{namespace="sock-shop"}
        )
      datasource:
      kind: PrometheusDatasource
      name: thanos-querier-datasource
0bed42a68343460b9f93e30c0974e3bc:
kind: Panel
spec:
display:
  name: Node.js heap utilization
plugin:
  kind: TimeSeriesChart
  spec: {}
queries:
  - kind: TimeSeriesQuery
    spec:
    plugin:
    kind: PrometheusTimeSeriesQuery
    spec:
      query: |
        100 *
        sum by (service) (
          nodejs_heap_size_used_bytes{namespace="sock-shop"}
        )
        /
        sum by (service) (
          nodejs_heap_size_total_bytes{namespace="sock-shop"}
        )
      datasource:
      kind: PrometheusDatasource
      name: thanos-querier-datasource
layouts:
  - kind: Grid
spec:
display:
  title: Node.js monitoring
  collapse:
```

```
        open: true
items:
  - x: 0
    "y": 6
    width: 7
    height: 5
    content:
      $ref: "#/spec/panels/e8d03e21e9894b0e923485ac55b5c0ef"
  - x: 12
    "y": 0
    width: 12
    height: 6
    content:
      $ref: "#/spec/panels/57f70808f6fe488cb3082c81a508f583"
  - x: 0
    "y": 0
    width: 12
    height: 6
    content:
      $ref: "#/spec/panels/0bed42a68343460b9f93e30c0974e3bc"
repeatVariable: ""
- kind: Grid
spec:
display:
  title: JVM monitoring
  collapse:
  open: true
items:
  - x: 0
    "y": 0
    width: 12
    height: 6
    content:
      $ref: "#/spec/panels/dc089e5df86140b09d0e9cda44ce17c8"
  - x: 12
    "y": 0
    width: 12
    height: 6
    content:
      $ref: "#/spec/panels/fe9b2a5b8d1843e4af6851f017b8e133"
  - x: 0
    "y": 6
    width: 12
    height: 6
    content:
      $ref: "#/spec/panels/38724d4938f34688b3cfe8348f592798"
repeatVariable: ""
- kind: Grid
spec:
display:
  title: CPU, Memory
  collapse:
  open: true
items:
```

```

      - x: 0
      "y": 0
      width: 12
      height: 6
      content:
      $ref: "#/spec/panels/3198885c174d49c990e238f64d2ec2dc"
      - x: 12
      "y": 0
      width: 12
      height: 6
      content:
      $ref: "#/spec/panels/d00a3bbc94644c11a7c442be5068ea66"
repeatVariable: ""
- kind: Grid
spec:
display:
  title: Application availability
  collapse:
  open: true
items:
  - x: 0
  "y": 0
  width: 24
  height: 12
  content:
  $ref: "#/spec/panels/8de86b66dc7641d8ac278acf175e3617"
repeatVariable: ""
- kind: Grid
spec:
display:
  title: " Latency"
  collapse:
  open: true
items:
  - x: 0
  "y": 0
  width: 24
  height: 8
  content:
  $ref: "#/spec/panels/6d648fbd3d424e4f98bf2848f5e15f5b"
  - x: 0
  "y": 8
  width: 24
  height: 8
  content:
  $ref: "#/spec/panels/57c197a19f4f4d1e81cd4af1631ee33c"
  - x: 0
  "y": 16
  width: 24
  height: 9
  content:
  $ref: "#/spec/panels/cf6bb5ba1541485db8ca61e255181258"
repeatVariable: ""
- kind: Grid

```

```

spec:
  display:
    title: " Traffic "
    collapse:
    open: true
  items:
    - x: 0
      "y": 0
      width: 24
      height: 9
      content:
        $ref: "#/spec/panels/2afe4ac11b0c40ffb09b09dd5b82328a"
    - x: 0
      "y": 9
      width: 7
      height: 6
      content:
        $ref: "#/spec/panels/424543212c9a47338d1377c4b34bfbac"
  repeatVariable: ""
  variables: []
  duration: 1h
  refreshInterval: 30s
  datasources: {}
  links: []

```

## 5.5 Metrics reference

The table below lists the metric names referenced while building this sample dashboard. The exact set available will differ from application to application.

| Metric name                  | Metric name                          | Metric name                      |
|------------------------------|--------------------------------------|----------------------------------|
| classes                      | go_memstats_stack_sys_bytes          | nodejs_heap_size_total_bytes     |
| classes_loaded               | go_memstats_sys_bytes                | nodejs_heap_size_used_bytes      |
| classes_unloaded             | heap                                 | nonheap                          |
| counter_status_200_metrics   | heap_committed                       | nonheap_committed                |
| gauge_response_metrics       | heap_init                            | nonheap_init                     |
| gc_g1_old_generation_count   | heap_used                            | nonheap_used                     |
| gc_g1_old_generation_time    | http_request_duration_seconds_bucket | process_cpu_seconds_total        |
| gc_g1_young_generation_count | http_request_duration_seconds_count  | process_cpu_system_seconds_total |
| gc_g1_young_generation_time  | http_request_duration_seconds_sum    | process_cpu_user_seconds_total   |
| go_gc_duration_seconds       | httpsessions_active                  | process_heap_bytes               |

|                                       |                                 |                                       |
|---------------------------------------|---------------------------------|---------------------------------------|
| go_gc_duration_seconds_count          | httpsessions_max                | process_max_fds                       |
| go_gc_duration_seconds_sum            | instance_uptime                 | process_open_fds                      |
| go_goroutines                         | jvm_classes_loaded              | process_resident_memory_bytes         |
| go_memstats_alloc_bytes               | jvm_classes_loaded_total        | process_start_time_seconds            |
| go_memstats_alloc_bytes_total         | jvm_classes_unloaded_total      | process_virtual_memory_bytes          |
| go_memstats_buck_hash_sys_bytes       | jvm_gc_collection_seconds_count | processors                            |
| go_memstats_frees_total               | jvm_gc_collection_seconds_sum   | request_duration_seconds_bucket       |
| go_memstats_gc_sys_bytes              | jvm_info                        | request_duration_seconds_count        |
| go_memstats_heap_alloc_bytes          | jvm_memory_bytes_committed      | request_duration_seconds_sum          |
| go_memstats_heap_idle_bytes           | jvm_memory_bytes_max            | scrape_body_size_bytes                |
| go_memstats_heap_inuse_bytes          | jvm_memory_bytes_used           | scrape_duration_seconds               |
| go_memstats_heap_objects              | jvm_memory_pool_bytes_committed | scrape_sample_limit                   |
| go_memstats_heap_released_bytes       | jvm_memory_pool_bytes_max       | scrape_samples_post_metric_relabeling |
| go_memstats_heap_released_bytes_total | jvm_memory_pool_bytes_used      | scrape_samples_scraped                |
| go_memstats_heap_sys_bytes            | jvm_threads_current             | scrape_series_added                   |
| go_memstats_last_gc_time_seconds      | jvm_threads_daemon              | scrape_timeout_seconds                |
| go_memstats_lookups_total             | jvm_threads_deadlocked          | systemload_average                    |
| go_memstats_mallocs_total             | jvm_threads_deadlocked_monitor  | threads                               |
| go_memstats_mcache_inuse_bytes        | jvm_threads_peak                | threads_daemon                        |
| go_memstats_mcache_sys_bytes          | jvm_threads_started_total       | threads_peak                          |
| go_memstats_mspan_inuse_bytes         | mem                             | threads_totalStarted                  |
| go_memstats_mspan_sys_bytes           | mem_free                        | up                                    |
| go_memstats_next_gc_bytes             | nodejs_active_handles_total     | uptime                                |
| go_memstats_other_sys_bytes           | nodejs_active_requests_total    |                                       |
| go_memstats_stack_inuse_bytes         | nodejs_eventloop_lag_seconds    |                                       |

## 5.6 Dashboard snapshots

### Node.js monitoring



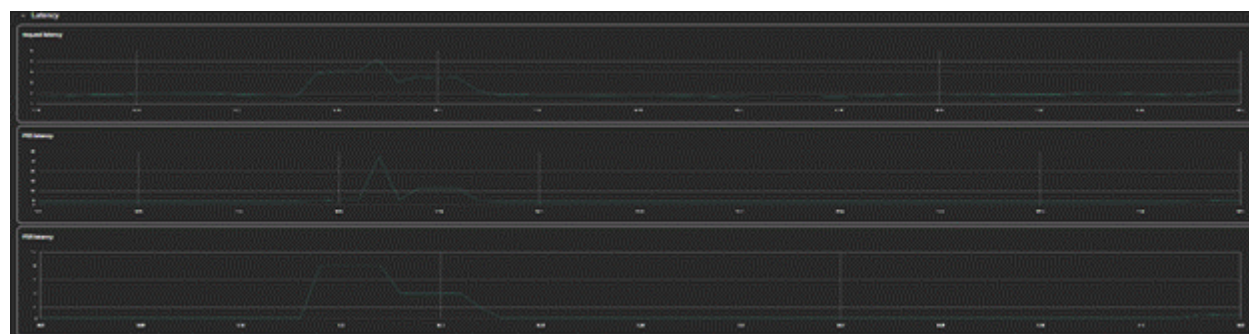
## JVM monitoring



## CPU and memory monitoring



## Latency



## Traffic

